# Rebuttal Response Round 1— Paper #571

"Automated Architecture-Aware Generation of Large Integer Multipliers on FPGAs"

Reviewer tags: **R1** (571A) **R2** (571B) **R3** (571C) **R4** (571D)

We thank all reviewers. We are encouraged that the work is seen as addressing "a genuine and practically important gap" that prior flows leave open (R4), as "The promised open-source release would be a useful artifact" (R3), with a "relevant IR optimizer and good results compared to related work"(R1). Below we group shared concerns first, then reviewer-specific points. We commit to every textual change mentioned.

## 1. Novelty: the contribution is not just in the tiler but the overall architecture-aware automation (R2, R3, R4)

R3 suggests framing the contribution "primarily as architecture-aware automation" rather than as multiple distinct algorithmic advances; R2 asks what the novelty is. **We agree with R3's framing and will adopt it.** The novelty is *not* a new tiler in isolation; it is that every step is shaped with the DSP architecture in mind to jointly maximize frequency *and* minimize scarce DSP usage, which no prior automated large-multiplier flow does:

- **Square Base Multiplier** scaling a square base turns the very few rectangular Karatsuba alignments (e.g. $26\times17 \Rightarrow 442k$) into dense ones ($44\times44 \Rightarrow 44k$), which is what *enables* deep Karatsuba which allows for DSP savings.
- **Separation of term generation from DSP mapping**, enabling DSP cascade-aware grouping (0-bit or 17-bit shifts), arithmetic rewrites (1-bit multiplications $\rightarrow$ AND, and $P1 + (P2 \ll n) \rightarrow \{P2, P1\}$ to zero-cost concatenate instead of accumulation). These saves expensive operations with cheaper ones, and saves LUTs by integerated post-adder in the DSP cascade instead of separate logic.
- **Pipeline-schedule-aware reduction** This is integrated into all the parts from the combinational logic to the internal DSP registers to the final output registers to allow automatically pipelining at the right points to break the combinational ceiling and reach high frequency. This is not just a "deep pipelining" step; it is a per-component scheduling that allows pipelining to be effective without excessive latency or resource overhead. This also enables the segmentation step that allows trading off latency for frequency.

We suggest the measurements show this is more than "pure engineering" (R2): the same arithmetic, mapped without architecture awareness (the HLS baseline in Table 2), runs at 298 MHz with *more* LUTs and DSPs than our 458–478 MHz designs.

## 2. What drives the gains? (and the iso-pipeline ablation) (R3 Q1, R4)

R4 and R3 ask whether the frequency gain comes from the arithmetic mapping or merely from deeper pipelining, and request an iso-pipeline ablation. **Deep pipelining is necessary but not sufficient**, and existing results separate the two effects:

1. **Base multiplier, single level, no deep tiling (Fig. 6, Table 1).** Before any hierarchical pipelining, our 77-bit base already beats the AMD IP: higher frequency, 2 fewer DSPs. This isolates *mapping* from *depth*.
2. **Resources, not just frequency.** Segment pipelining cannot reduce DSP count; only the decomposition/mapping can. At 1024-bit, other works use 17–45% more DSPs than our proposed approach — a result pipelining alone cannot produce.
3. **Per-component attribution:**
   - *Base selection* sets the DSP floor and the achievable frequency (scaling a slow/wasteful base cannot recover either).
   - *Hierarchical decomposition* controls how deeply Karatsuba removes multipliers (Table 2: more Karatsuba levels $\rightarrow$ fewer DSPs).
   - *Cascade-aware mapping* cuts LUTs used for logic (§3.2) and raises frequency.
   - *Optimizer rewrites* trade eligible expensive operations to cheaper ones, such as replacing 1-bit multiplications with AND gates or using zero-cost concatenations instead of accumulation saving resources and critical path delay.
   - *Segmentation* (Table 3) choosing the combination of segment size and pipeline depth allows trading off latency for frequency.
4. **The requested cross-mapping ablation is approximated by the HLS row.** "IMpress mapping + our pipelining" is not runnable by us — IMpress is HLS, and we cannot re-pipeline a competitor's closed flow at any granularity without their generator. The honest iso-tool point already in the paper is the HLS baseline: same arithmetic left to the tool, far worse frequency *and* more resources. We will state this limitation explicitly and add the per-component breakdown above as a short ablation paragraph.

**On the "4–5× pipeline depth"** (R3, R4): high frequency requires depth, but Table 3 shows depth is a *tunable*

*knob*, not a fixed cost — a 256-bit segment cuts latency 100→58 cycles while still beating prior work on frequency. We will frame this tradeoff more explicitly.

## 3. Single-vendor / cross-platform generality (R1, R3, R4)

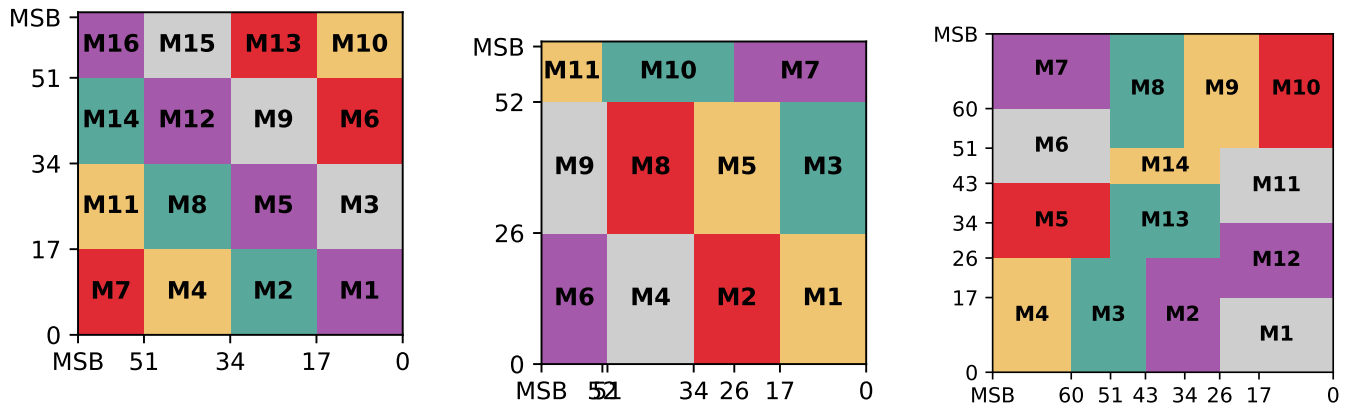
Three reviewers ask about generality of our approach beyond AMD UltraScale+. ⇒ **The flow is vendor-agnostic by construction; only the leaf instantiation templates are architecture-specific.** The Versal/DSP58 result (Table 2) is the evidence: moving from a 27×18 to a 27×24 primitive, the tiler, hierarchy generator, untimed IR, software simulation, scheduling, and RTL/testbench generation are all *reused unchanged* — only the DSP instantiation template and board constraints differ. Porting to Intel/Altera Stratix/Agilex therefore requires authoring the DSP instantiation template, project/reporting templates, and constraint files, not rewriting the algorithms (R2's "what is assumed"). We will state this porting checklist explicitly and soften "broadly applicable" to "architecture-parameterized, demonstrated on two AMD DSP generations"

On R1's Altera specifics:

- *Square primitives.* Stratix 10 has a 27×27 hard multiplier, so the primitive is already square allowing immediate Karatsuba alignment. The only difference between the flow for AMD and Intel is only the hardware primitive instantiation template; the rest of the flow is unchanged.
- *Hard fixed-point cores.* Stratix 10 has hard fixed-point cores that can be used for multiplication, but they are not relevant to wide integer multiplication.

## 4. Architectural difference vs. the AMD IP (R1)

R1 asks how our design is architecturally different from the AMD IP (not just in final resources). ⇒ **We tried to conclude the decomposition of the 64-bit IP from the device view.** It instantiates 16 DSPs with 15 PCOUT→PCIN cascade connections, i.e. *fully cascaded design*, which means that all inter-tile shifts are 0 or 17. This means that the IP caps each DSP to $\leq 17\times17$ (we confirmed 17×17, 17×13, 17×18 tiles via reporting non-fixed port widths in the dsp block through Tcl console), implying a $(17, 17, 17, 13)$ decomposition. Replicating it reproduces its results to within ≈100 LUTs / 50 FFs (shift logic); With this decomposition, the base multiplier IP is wasteful, as it does not use the full 27×18 capability of the DSPs. If we had chosen schoolbook decomposition, we would have used only 11 DSPs instead of 16 with the decomposition shown in Fig. 1b. When scaling to 1024-bit, this would still be using more DSPs than our 43-bit base, but it will be much more optimal than using the direct IP. If we stick to the same decomposition, we can use Karatsuba to save 4 of the 16 DSPs, but it will be wasteful especially when scaling to larger sizes, and this is shown in Fig.7. This is precisely the architecture-aware advantage. We will add this analysis and the figures.




(a) IP's $(17, 17, 17, 13)$ decomposition (16 DSPs).

(b) Our Schoolbook mapping (11 DSPs).

(c) Our 78-bit base (14 DSPs).

Figure 1: (Fig. A) AMD IP vs. our architecture-aware mappings.

## 5. FFs and the normalized-area metric (R3)

R3 notes $A_{\text{norm}} = \#\text{LUTs} + 126 \cdot \#\text{DSPs}$ excludes FFs while our designs use 207K–274K FFs. ⇒ **We just used the exact metric of the prior work we compare to**. FFs are excluded in all the works we denote as N/A Anyways, in these devices FFs are the most abundant, rarely the binding resource, whereas DSPs are scarce. Our design philosophy also provide a tradeoff knob to reduce FFs at the cost of frequency (Table 3), so including FFs in the metric would

shift the tradeoff curve We agree the asymmetry should be visible and will (i) add an FF column for our designs, (ii) report the prior works' FFs as N/A only because *they do not report them* (**R3** Q on N/A), and (iii) add one sentence noting that an FF-inclusive metric would shift the tradeoff but not the DSP-and-frequency advantage, which is the scarce-resource axis.

## 6. Table 1 "LUT inversion" at 78-bit (R4)

R4 asks why 78-bit one-group (537 LUTs) exceeds multi-group (429), contradicting "one-group uses fewer LUTs," and whether counts are post-synthesis with SRL inference. ⇒ **All results reported in the paper are post place and route, and LUTs used in the one-group strategy are Shift-Register LUTs, not logic LUTs.** The exact count is as following: The one-group strategy uses 0 logic LUTs while the multi-group strategy uses 62 and 239 logic LUTs for 44-bit and 78-bit respectively. **We will split the LUT column into logic-LUTs and SRL-LUTs in Table 1** to make this unambiguous.

## 7. Missing comparisons and the §3.2 vs. Table 2 numbers (R3 Q2, R4)

**ILP [17] and rectangular-Karatsuba [16]** (**R3** Q2/comment 2): the ILP is the *predecessor* of beam-search, which we already report and beat, thus beat the ILP. The rectangular-Karatsuba work targets a different regime: its largest reported case is 112×120, and it has no hierarchical decomposition, so it cannot scale to the 1024-bit-and-up "large integer" regime we target (their "large" means > 64-bit). We will add this explanation in §5 with this explicit justification.

**OpenNTT row** (R4): NTT is a different algorithmic class with a different latency regime (689 cycles); we include it only for context. We completed the fields available from their paper and clearly label it as a non-tiling reference rather than a comparison.

**Reconciling §3.2 (464/507/516) with Table 2 (810)** (**R3** comment 3): these report *different filling strategies*. The 464/507/516 are DSP counts for only remainder-filling parts denoting different strategies (cascade-compatible vs. greedy vs. non-cascade). Table 2 reports fully-implemented 1024-bit lowest-DSP configuration not just the remainder-filling strips.

## 8. Headline percentages (R3)

**R3** is correct. ⇒ The abstract/caption figures are all vs. IMpress2 ( what we believe to be the strongest prior approach); §4.1 is the per-baseline detail. We also interchangeably used "17% fewer DSPs (ours)" with "17% more DSPs (theirs)". **We will make every percentage consistent, state the baseline inline, and correct the direction of the DSP comparison.**

## 9. Remaining points (R1, R2)

**Experimental setup** (R1): EPYC 7763 (64C), 512 GB RAM, Ubuntu 20.04, Vivado 2022.1, AMD Alveo U280; all results are post-place-and-route. We will add this to the evaluation section.

**Why not leave it to HLS / native tools?** (R2): because they underperform on this problem; our own HLS baseline (Table 2) runs at 298 MHz with more LUTs and DSPs, since the tools do not exploit cascade paths, asymmetric ports, or per-DSP pipeline depth for wide structured arithmetic.

**ML design-space-exploration work** (R2): TODO We believe these clarifications and additions (per-component ablation, FF column, logic/SRL LUT split, IP architectural analysis with figures, explicit ILP/Kumm/OpenNTT justification, generality/porting checklist, corrected percentages, and full setup) directly address every concern. We thank the reviewers for their feedback and for helping us clarify the presentation and strengthen the paper.